



RoboCup Junior — Rescue Line

TEAM FIELD GUIDE · SOFTWARE EDITION

Rescue Robot Code & Field Manual

Everything you need to run the robot, push code changes to the Pi, and understand what every file actually does — written so nobody has to guess.

Raspberry Pi 5 · rescue.local · Written for the whole team, not just the coders

• Table of Contents

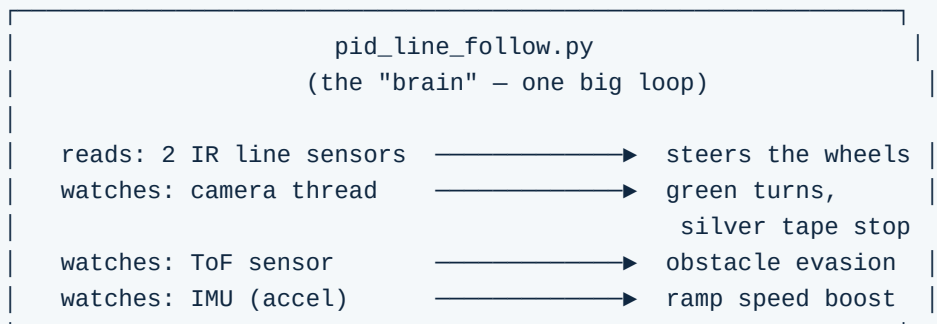
- 1 The Big Picture — what this robot actually does
- 2 Connecting to the Pi (SSH) & starting the .venv
- 3 Running the robot
- 4 Sending code from your laptop to the Pi (scp)
- 5 Editing code directly on the Pi (nano)
- 6 File structure on the Pi
- 7 What every file does (plain English)
- 8 Playground scripts — things you can test and tweak
- 9 Troubleshooting & cheat sheet

HOW TO USE THIS GUIDE

You don't need to read this front to back. If you just need to push a code change, jump to Section 4. If something's broken, jump to Section 9. If you're trying to understand what a specific file does, Section 7 has one card per file.

1 The Big Picture

The robot follows a black line using two little sensors underneath it, and uses its camera for everything else: spotting green direction arrows, spotting the silver "evacuation zone" tape, and double-checking obstacles. Here's the mental model:



The hardware, in one table

Part	What it does	Connects via
2x IR line sensors	See black line vs. white floor	GPIO 24 (left) / GPIO 23 (right)
2x drive motors (front only)	Move the robot — rear motors are disconnected, it's 2-wheel drive	I2C (0x20 left, 0x19 right)
Pi Camera	Sees green arrows, silver tape, and confirms obstacles	Camera ribbon cable
ToF distance sensor	Detects something close in front (obstacle or ramp)	I2C (address 0x51)
BNO085 IMU	Feels tilt/acceleration — used to detect ramps	I2C (address 0x4A)
9g camera-pan servo	Tilts the camera up to look at evacuation zone stuff	GPIO 18 (pulse signal)
Parallax continuous servo	Lifts/lowers the rear claw	GPIO 19 (pulse signal)
9g claw open/close servo	Opens and closes the claw's grip to grab/release a victim	GPIO — pin TBD (pulse signal)

THREE SERVOS, THREE JOBS — DON'T MIX THEM UP

It's easy to lump "the servos" together, but there are three separate ones doing three separate things: the **camera-pan** servo tilts the camera up/down, the **claw-lift** servo (Parallax, continuous-rotation) raises/lowers the whole claw arm, and the **claw open/close** servo opens and closes the claw's grip itself. Each has its own GPIO pin and its own test script (Section 8).

GOLDEN RULE

The line sensors are spaced apart on purpose. When the robot is perfectly centered on the line, **both** sensors see white — that's normal, not an error! Only one sensor sees black when the robot has drifted

enough to catch the line's edge. Keep this in mind whenever you're reading the code.

2 Connecting to the Pi & Starting the .venv

Step 1 — Connect over SSH

Open a terminal (PowerShell on Windows, Terminal on Mac) and type:

```
ssh niges@rescue.local
```

This is like remote-controlling the Pi's keyboard from your laptop. Type the Pi's password when asked (ask a teammate if you don't have it).

WHAT IS RESCUE.LOCAL?

It's the nickname (hostname) the Pi answers to on the wifi network. As long as your laptop and the Pi are on the same network, this name works instead of typing an IP address.

Step 2 — Move into the project folder

```
cd ~/rescue_robot
```

Step 3 — Turn on the Python virtual environment (.venv)

The `.venv` is a private little bubble that holds all the Python libraries this project needs (camera stuff, motor drivers, etc.) without messing with the rest of the Pi. You **must** turn it on before running any of our scripts, every single time you open a new terminal.

```
source .venv/bin/activate
```

HOW DO I KNOW IT WORKED?

Your terminal prompt will change to show `(.venv)` at the start of the line. If you don't see that, the scripts below will fail with "module not found" errors.

Step 4 (only if .venv doesn't exist yet)

This should already be set up on the Pi, but if you ever need to build it from scratch:

```
python3 -m venv .venv --system-site-packages
source .venv/bin/activate
pip install -r requirements.txt
pip install adafruit-circuitpython-bno08x adafruit-blinka
pip install git+https://github.com/Aw3someAndrew/SteelBar_CircuitPython_powerful_bldc_driv
```

DON'T FORGET --SYSTEM-SITE-PACKAGES

The camera library (`picamera2`) can't be installed with pip — it has to come from the Pi's system packages (`sudo apt install python3-picamera2`). The `--system-site-packages` flag lets the

venv "see" it. If you forget this flag, the camera code won't import.

3 Running the Robot

1 SSH in and activate the `venv` (see Section 2).

2 Run the main program:

```
python3 line_follow/pid_line_follow.py
```

This is *the* file — the entire robot behaviour (line following, green arrow turns, silver-tape stop, obstacle evasion, ramp speed boost) all happens inside this one script.

3 **Watch the terminal.** It prints out what's happening live — e.g. `"obstacle detected -- running evade sequence"` or `"possible spill detected -- stopping to verify..."` — so you can see what the robot thinks it's doing.

4 **To stop it,** press `Ctrl + C` in the terminal. The robot will stop its motors cleanly.

THE PHYSICAL SWITCH ALSO WORKS

There's a physical switch on the robot (GPIO 21) that a background service (`switch_watcher.py`) is always watching. Flip it ON and it automatically launches `pid_line_follow.py` for you — flip it OFF and it stops. Handy at competition when you don't want to be tethered to a laptop.

What actually happens when it runs

Sees	Robot does
Line drifting left/right under a sensor	Steers back toward center (the core PID loop)
Green arrow marker on camera	Ignores the line sensors briefly and turns that direction
Silver tape on camera	Stops, double-checks for 1 second, then stops for good if confirmed
Something within 50mm on the ToF sensor	Asks the camera to confirm it's really a bottle, then backs up + swerves around it
IMU senses it's nosing upward (ramp)	Speeds up by 50% until it senses level ground again

4 Sending Code From Your Laptop to the Pi

When you edit a file **on your laptop** (in the `rescue_robot` folder), the Pi doesn't automatically see the change. You have to copy it across using `scp` (secure copy) — think of it as "upload this one file to the Pi."

The pattern (Windows PowerShell)

```
scp "C:\Users\RAM0038\OneDrive - Melbourne High School\Desktop\rescue_robot\line_follow\FILENAME.py" pi:/rescue_robot/line_follow/
```

KEEP THE QUOTES!

Because the OneDrive folder path has spaces in it, you must wrap the laptop-side path in double quotes exactly like the example above, or PowerShell will get confused about where the path ends.

Copy the whole `line_follow` folder in one go

Instead of scp-ing files one at a time, this pushes everything in `line_follow/` at once — much faster after a big batch of changes:

```
scp -r "C:\Users\RAM0038\OneDrive - Melbourne High School\Desktop\rescue_robot\line_follow" pi:/rescue_robot/line_follow/
```

Going the other way (Pi → laptop)

If you changed something directly on the Pi and want a copy back on your laptop:

```
scp niges@rescue.local:~/rescue_robot/line_follow/FILENAME.py "C:\Users\RAM0038\OneDrive\rescue_robot\line_follow\FILENAME.py"
```

MAKE SURE THE FOLDER EXISTS FIRST

If you're pushing to a brand new folder that's never existed on the Pi, create it first or scp will fail:

```
ssh niges@rescue.local "mkdir -p ~/rescue_robot/line_follow"
```

RULE OF THUMB

Laptop = where you write and edit code with a proper editor. Pi = where the code actually runs on the robot. `scp` is the bridge between the two. After every `scp`, it's a good habit to re-run the robot to check nothing broke.

5 Editing Code Directly on the Pi

Sometimes it's faster to make a tiny tweak (like changing one number) directly on the Pi instead of editing on your laptop and scp-ing it over. For this we use **nano**, a simple text editor that runs inside the terminal.

- 1 SSH into the Pi and go to the file's folder (see Section 2).
- 2 Open the file:

```
nano line_follow/obstacle_evade.py
```

- 3 Use the arrow keys to move around and edit like a normal text editor. No mouse needed — just type.
- 4 When you're done, save and exit:
 - Press `Ctrl + O` (that's the letter O, for "output/write out"), then `Enter` to confirm the filename
 - Press `Ctrl + X` to exit nano

CAREFUL — THIS ONLY EXISTS ON THE PI!

A change you make directly on the Pi with nano will **not** appear on anyone's laptop or in the GitHub repo unless you scp it back (Section 4, "Pi → laptop"). It's easy to lose good changes this way — if you make a real improvement on the Pi, pull it back to your laptop and commit it to GitHub as soon as you can.

WHEN NANO IS THE RIGHT CALL

Great for quick number tweaks (like changing `BASE_SPEED = 30.0` to try a different speed) right before a run. Not great for big rewrites — do those on a laptop with a real editor, then scp them over.

6 File Structure on the Pi

The Pi's copy of the project lives at `~/rescue_robot` (which is short for `/home/nigesh/rescue_robot`) and mirrors your laptop's folder:

```
~/rescue_robot/
├── line_follow/                ← ACTIVE robot code – this is what matters
│   ├── pid_line_follow.py      ← ENTRY POINT – run this to start the robot
│   ├── camera_watcher.py       ← shared camera thread (green/silver/frame-grab)
│   ├── green_detector.py       ← standalone green-marker test version
│   ├── spill_classifier.py      ← silver tape detector (trained model)
│   ├── spill_model_weights.py  ← the trained model's numbers
│   ├── tof_sensor.py           ← raw distance-sensor driver
│   ├── tof_detector.py         ← background thread polling the distance sensor
│   ├── bottle_detector.py      ← camera check: "is that really a bottle?"
│   ├── obstacle_evade.py       ← the back-up-and-swerve maneuver
│   ├── imu_sensor.py           ← raw accelerometer/gyro/compass readings
│   ├── accel_speed_boost.py    ← background thread: ramp speed boost
│   ├── camera_servo.py         ← camera-tilt servo control
│   ├── calibrate_servo.py      ← interactive servo calibration tool
│   ├── test_claw_open_close.py ← claw open/close servo test (pin TBD)
│   └── test_*.py               ← PLAYGROUND scripts, see Section 8
├── control/
│   └── motors.py                ← talks to the two drive motors
├── sensor_tests/               ← low-level line-sensor diagnostics
├── old_camera_version/         ← ARCHIVED, ignore this, not used anymore
├── switch_watcher.py           ← watches the physical on/off switch
├── rescue-robot-switch.service ← makes switch_watcher.py start on boot
├── requirements.txt            ← list of Python libraries needed
├── setup_pi.sh                 ← one-time Pi setup script
└── README.md / FILE_MAP.md     ← written docs, more technical than this PDF
```

WHERE AM I ON THE PI RIGHT NOW?

Type `pwd` (print working directory) any time you're SSH'd in and unsure which folder you're standing in.
Type `ls` to list what's in the current folder.

7 What Every File Does

Plain-English explanations, one card per file. **CORE** = part of the main running system. **HELPER** = supporting code used by core files. **PLAYGROUND** = standalone script for testing/tuning — see Section 8 for the full list of these.

`pid_line_follow.py`

CORE

The entry point — run this to start the robot. It's one big loop that runs 50 times a second. Each time through the loop it: checks the distance sensor for obstacles, checks the camera for silver tape and green arrows, reads the two line sensors, and decides how fast to spin each wheel. Everything else in `line_follow/` exists to feed information into this one file.

The line-following math (inside `pid_line_follow.py`)

CORE

Each line sensor gives a 0 (white) or 1 (black). `error = left_sensor - right_sensor`, so error is always **-1, 0, or +1**. That error gets multiplied by `Kp` (how hard to react right now) plus a bit based on how the error is *changing* (multiplied by `Kd` — this smooths out overcorrecting). The result is added to one wheel's speed and subtracted from the other, so the robot leans back toward center. Remember: both sensors reading 0 (white) means "perfectly centered," not "lost" — so there's no separate lost-line safety net, the math just naturally drives straight.

`camera_watcher.py`

CORE

The robot only has **one** camera, but three different things need to look at it (green markers, silver tape, obstacle bottle-checking). This file runs the camera *once* in the background and hands out the latest picture to whoever asks — like a librarian who fetches one book that three people take turns reading, instead of everyone fighting over the shelf.

`tof_sensor.py` + `tof_detector.py`

HELPER

`tof_sensor.py` talks to the physical distance sensor (ToF = "Time of Flight" — it bounces light off things to measure distance in millimetres). `tof_detector.py` just keeps asking it "how far away is the nearest thing?" in the background so `pid_line_follow.py` always has a fresh answer ready without waiting.

`bottle_detector.py`

HELPER

The distance sensor can't tell the difference between "there's a bottle in front of me" and "there's a ramp sloping up in front of me" — both just look like "something close." So when the distance sensor triggers, this file takes a look at the camera picture and checks: does the shape look like a bottle (mostly straight up-and-down edges) or a ramp (mostly diagonal/sideways edges)? Only if it looks like a bottle does the robot bother swerving.

obstacle_evade.py

CORE

Once a bottle is confirmed, this runs a fixed sequence of moves to get around it: **back up** → **turn right** → **drive forward** → **turn left** → **drive straight** → **turn left** → **turn right**, then hands control back to normal line following. Every distance/turn amount is its own labelled variable at the top of the file (see Section 8) so it's easy to tune without digging through code.

imu_sensor.py

HELPER

Talks to the IMU (a chip that senses acceleration, rotation, and magnetic direction) and hands back the raw numbers — nothing fancy, just "here's what the sensor currently reads." On flat ground, the up/down (Y) reading sits around `-9.8` because of gravity.

accel_speed_boost.py

CORE

Watches the IMU's Y reading in the background. If it drops below `-15`, that means the robot is nosing upward — i.e. it's driving onto a ramp — so this tells `pid_line_follow.py` to speed up by 50% until the reading comes back to normal.

spill_classifier.py + spill_model_weights.py

CORE

Detects the silver evacuation-zone tape. Plain colour/brightness checks didn't work reliably (the tape and the tile floor look too similar), so this uses a small trained model instead — it was "shown" lots of example photos of tape vs. floor and learned the difference. `spill_model_weights.py` just stores the numbers that model learned; you shouldn't need to touch it unless retraining.

green_detector.py

HELPER

A standalone, simpler version of the green-arrow detection logic that `camera_watcher.py` now handles as part of the main loop. Kept around for quick isolated testing — not used when the robot is actually running.

control/motors.py

CORE

The only file that actually talks to the drive motors. Converts a "-100% to +100% speed" number for each wheel into the exact electrical commands the motor controllers understand. If the robot drives the wrong direction or won't move at all, this is the file to check.

camera_servo.py + calibrate_servo.py

HELPER

Controls the small 9g servo that tilts the camera up/down. `calibrate_servo.py` is an interactive tool for finding the exact pulse widths that correspond to "look forward" and "look up" for this specific servo (every servo is a little different).

test_claw_open_close.py

PLAYGROUND

Tests the third servo on the robot — the one that opens and closes the claw's grip (separate from the claw-lift servo that raises the whole arm). It's a positional servo like the camera one, not a continuous-

rotation one. **The GPIO pin isn't finalised yet** — there's a clearly marked `CLAW_SERVO_PIN` placeholder near the top of the file to update once it's wired up.

switch_watcher.py + rescue-robot-switch.service

CORE

Watches the physical on/off switch on the robot. Flip it on and it automatically launches `pid_line_follow.py` for you; flip it off and it stops. The `.service` file just tells the Pi to start watching the switch automatically every time it boots up.

8 Playground Scripts

These are small, standalone scripts you can run on their own (not part of the main robot loop) to test one specific thing and play with numbers. Every one of them has a variable near the top clearly marked with a comment telling you what to change.

test_servo.py

PLAYGROUND

Tests the 9g camera-pan servo. Run it, change `DURATION_S` at the top, run it again, and watch how much further/less the servo turns.

```
python3 line_follow/test_servo.py
```

test_claw_servo.py

PLAYGROUND

Tests the claw-lift servo (a different kind — this one spins continuously rather than moving to an angle). Change `PULSE_US` (1300 = up, 1500 = stop, 1700 = down) and `DURATION_S` to control how far it travels.

```
python3 line_follow/test_claw_servo.py
```

test_claw_open_close.py

PLAYGROUND

Tests the claw **open/close** servo (grips/releases a victim — not the same servo as the one above that lifts the whole arm). Change `OPEN_PULSE_US` and `CLOSE_PULSE_US` until the claw fully opens and fully closes without straining.

```
python3 line_follow/test_claw_open_close.py
```

PIN NOT SET YET

`CLAW_SERVO_PIN` at the top of the file is a placeholder — update it to the real GPIO pin once this servo is wired in, and make sure it doesn't clash with GPIO 18 (camera) or GPIO 19 (claw lift).

test_tof_distance.py

PLAYGROUND

Just prints the distance sensor's reading over and over, in millimetres. Handy for figuring out exactly how far away something needs to be before the robot should react to it. Press Ctrl+C to stop.

```
python3 line_follow/test_tof_distance.py
```

test_180_spin.py

PLAYGROUND

Practises the "spin 180° and back up to grab something" move (for the evacuation zone, since the claw is on the back of the robot). Set `DISTANCE_MM` to how far away the target was, then tune `TURN_DURATION_S` until it lands on a clean half-turn, and `MM_PER_SECOND` to match how fast the robot actually reverses.

```
python3 line_follow/test_180_spin.py
```

test_forward.py PLAYGROUND

Drives both wheels forward briefly. Use this after the robot has been powered off/on to check the motors are still calibrated correctly and driving straight.

```
python3 line_follow/test_forward.py
```

calibrate_servo.py PLAYGROUND

Interactive — use the keyboard to nudge the camera servo in tiny steps and mark down its center/min/max positions. Use this once per servo, not every time.

```
python3 line_follow/calibrate_servo.py
```

**test_green_webcam.py / test_spill_webcam.py /
test_spill_classifier_webcam.py** PLAYGROUND

Test the camera detection logic using a laptop webcam instead of the Pi's camera — useful for tweaking detection without needing the whole robot powered up.

Obstacle-evasion tunable variables (obstacle_evade.py)

This is the most likely file you'll want to "play around in" without breaking anything — every step of the swerve-around-a-bottle maneuver has its own named variable:

Variable	Controls
<code>EVADE_SPEED</code>	Speed (%) for straight moves (back up / drive forward)
<code>TURN_SPEED</code>	Speed (%) for turning-in-place moves
<code>BACK_DURATION_S</code>	How long it backs away from the obstacle first
<code>RIGHT_TURN_1_DURATION_S</code>	How long the first right turn lasts
<code>FORWARD_1_DURATION_S</code>	How far forward it drives around the obstacle
<code>LEFT_TURN_1_DURATION_S</code>	How long the first left turn lasts
<code>FORWARD_2_DURATION_S</code>	How far forward it drives past the obstacle
<code>LEFT_TURN_2_DURATION_S</code>	How long the second left turn lasts
<code>RIGHT_TURN_2_DURATION_S</code>	How long the final right turn (straightening up) lasts

HOW TO TUNE IT

Longer duration at the same speed = more distance travelled / more degrees turned. Change one number at a time, scp it over (Section 4), run the robot near an obstacle, and watch what changes.

9 Troubleshooting & Cheat Sheet

Quick command reference

What you want to do	Command
Connect to the Pi	<code>ssh niges@rescue.local</code>
Turn on the venv	<code>source .venv/bin/activate</code>
Run the robot	<code>python3 line_follow/pid_line_follow.py</code>
Stop the robot	<code>Ctrl + C</code>
Edit a file on the Pi	<code>nano line_follow/FILENAME.py</code>
Save & quit nano	<code>Ctrl+O</code> , Enter, then <code>Ctrl+X</code>
Copy laptop → Pi	<code>scp "laptop\path\file.py"</code> <code>niges@rescue.local:~/rescue_robot/line_follow/file.py</code>
Copy Pi → laptop	<code>scp niges@rescue.local:~/rescue_robot/line_follow/file.py</code> <code>"laptop\path\file.py"</code>
See what's in a folder	<code>ls</code>
See which folder you're in	<code>pwd</code>

Common problems

"MODULENOTFOUNDERROR" WHEN RUNNING A SCRIPT

You almost certainly forgot to activate the venv. Run `source .venv/bin/activate` first — your prompt should show `(.venv)` .

ROBOT DRIVES CROOKED OR WON'T MOVE RIGHT AFTER POWER-ON

The motor calibration numbers in `control/motors.py` can reset when the motor driver boards lose power. Run `test_forward.py` to check — if it's still crooked, someone will need to re-calibrate and update `MOTOR_CONFIG` in that file.

SCP SAYS "NO SUCH FILE OR DIRECTORY"

Double-check the path is wrapped in quotes and spelled exactly right (capitalisation matters on the Pi side!). If the destination folder doesn't exist yet on the Pi, create it first: `ssh niges@rescue.local "mkdir -p ~/rescue_robot/line_follow"` .

CAMERA-RELATED CODE WON'T IMPORT

Check the venv was created with `--system-site-packages` (Section 2, Step 4) — `picamera2` has to come from the system, not pip.

MADE AN EDIT DIRECTLY ON THE PI WITH NANO — DON'T LOSE IT!

Anything typed straight into the Pi only exists there until you scp it back to a laptop (Section 4). If it's a real improvement, copy it back and get it into GitHub as soon as you can — otherwise it'll quietly vanish next time someone overwrites that file from their own laptop copy.

STILL STUCK?

Ask in the team chat with exactly what command you ran and exactly what error message showed up — a screenshot of the terminal is even better. "It doesn't work" is much harder to help with than "I ran `python3 line_follow/pid_line_follow.py` and got `ModuleNotFoundError: No module named 'lgpio'`."